SAYLANI IT TRAINING

SQL Server — Class 2 Homework

Joins & Views

BikeStores Sample Database

Instructions

Write a SQL Server query to answer each question below using the BikeStores sample database.

Tables: production.products | production.brands | production.categories | sales.customers | sales.orders | sales.order_items | sales.staffs | sales.stores | sales.stocks

Topic Legend:

INNER JOIN	LEFT JOIN	RIGHT JOIN	SELF JOIN	VIEWS	BONUS

Q1 Show all product names along with their brand name. Sort by brand name, then by product name alphabetically. — INNER JOIN

Your answer:

```
select
        b.brand_name,
        p.product_name
from [production].[products] as p
inner join [production].[brands] as b
on p.brand_id = b.brand_id
order by b.brand_name asc, p.product_name asc;
```

Q2 List all products with their category name and list price. Sort by category name, then by price from cheapest to most expensive. — INNER JOIN

Your answer:

```
select
        p.product_name,
        c.category_name,
        p.list_price
from [production].[products] as p
inner join [production].[categories] as c
on c.category_id = p.category_id
order by c.category_name asc, p.list_price asc;
```

Q3 Show all orders with the customer's full name and order date. Sort by order date from newest to oldest. **INNER JOIN**

Your answer:

```
select
        c.first_name + ' '+ c.last_name as customers_full_name,
        o.order_date,
        o.order_id
from [sales].[orders] as o
inner join [sales].[customers] as c
on o.customer_id = c.customer_id
order by o.order_date desc
```

Q4 Display each order item with the product name, quantity, unit price, and a computed column called "Line Total" (quantity × list_price). Sort by order ID. **INNER JOIN**

Your answer:

```
select
                oi.order_id,
                p.product_name,
                oi.quantity,
                p.list_price as Unit_Price,
                oi.quantity * p.list_price as Line_Total
from [production].[products] as p
inner join [sales].[order_items] as oi
on p.product_id = oi.product_id
order by order_id asc;
```

Q5 Show each order along with the store name where it was placed and the order date. Sort by store name. **INNER JOIN**

Your answer:

```
select
                s.store_name,
                s.city,
                o.order_date
from [sales].[stores] as s
inner join [sales].[orders] as o
on s.store_id = o.store_id
order by s.store_name asc;
```

Q6 Show each order with: order ID, customer full name, store name, and the staff member's full name who handled it. **INNER JOIN**

Your answer:

```
select
                o.order_id,
                c.first_name +' '+ c.last_name as Customer_Name,
                s.store_name,
                staff.first_name +' '+ staff.last_name as Staff_Full_Names
from [sales].[orders] as o
inner join [sales].[customers] as c
on o.customer_id = c.customer_id
inner join [sales].[stores] as s
on o.store_id = s.store_id
inner join [sales].[staffs] as staff
on o.staff_id = staff.staff_id
```

Q7 List all products from the brand "Trek" along with their category name and price. Sort by price descending. (Use JOIN — do NOT filter by brand_id directly.) **INNER JOIN**

Your answer:

```
select
                p.product_name,
                c.category_id,
                p.list_price
from [production].[products] as p
inner join [production].[categories] as c
on p.category_id = c.category_id
inner join [production].[brands] as b
on p.brand_id = b.brand_id
where b.brand_name = 'Trek'
order by p.list_price desc ;
```

Q8 Find all customers from the state of "NY" who have placed at least one order. Show customer full name, city, and order date. (Use JOIN — do not use a subquery.) **INNER JOIN**

Your answer:

```
select
                c.first_name +' '+ c.last_name as Customer_Name,
                c.city,
                o.order_date
from [sales].[customers] as c
inner join  [sales].[orders] as o
on c.customer_id = o.customer_id
where c.state= 'NY'
```

Q9 Show all completed orders (order_status = 4) from the store "Rowlett Bikes". Display order ID, customer full name, and order date. **INNER JOIN**

Your answer:

```
select
		o.order_id,
		c.first_name +' '+ c.last_name as Customer_Name,
		o.order_date

from [sales].[orders] as o
inner join [sales].[customers] as c
on o.customer_id = c.customer_id
inner join [sales].[stores] as s
on o.store_id = s.store_id
where o.order_status = '4' and s.store_name= 'Rowlett Bikes'
```

Q10 List ALL customers and any orders they have placed. Include customers who have never placed an order (show NULL for order columns). Sort by customer ID. **LEFT JOIN**

Your answer:

```
 select
		c.customer_id,
		c.first_name +' '+ c.last_name as Customer_Name,
		o.order_id,
		o.order_date
from [sales].[customers] as c
left join [sales].[orders] as o
on c.customer_id = o.customer_id
order by c.customer_id asc;
```

Q11 Find all customers who have NEVER placed an order. Show their full name and email. **LEFT JOIN**

Your answer:

```
 select
		c.first_name +' '+ c.last_name as Customer_Name,
		c.email
from [sales].[customers] as c
left join [sales].[orders] as o
on c.customer_id = o.customer_id
where o.order_id is Null
```

Q12 List all products and their stock quantity at every store. Include products that have NO stock record at all. Show product name, store ID, and quantity.

LEFT JOIN

Your answer:

```
select
                p.product_name,
                s.store_id,
                s.quantity
from [production].[products] as p
left join [production].[stocks] as s
on p.product_id = s.product_id
```

Q13 Find all products that have NEVER been ordered (no record in order_items). Show product name and list price.

LEFT JOIN

Your answer:

```
select
                p.product_name,
                p.list_price
from [production].[products] as p
left join [sales].[order_items] as oi
on p.product_id = oi.product_id
where oi.order_id is null
```

Q14 List each staff member along with the full name of their manager. Staff with no manager (top-level) should still appear — show NULL for manager name.

SELF JOIN

Your answer:

```
select
                staff.first_name +' '+staff.last_name as staff_Name,
                manager.first_name +' '+ manager.last_name as Manager_Name

from [sales].[staffs] as staff
left join [sales].[staffs] as manager
on staff.manager_id = manager.staff_id
```

Q15 Create a view called vw_bike_catalog that shows product_name, brand_name, category_name, model_year, and list_price. Then query it to show only products priced over $2,000, sorted by price descending.

VIEWS

Your answer

```
create view vw_bike_catalog as
select
                    p.product_name,
                    b.brand_name,
                    c.category_name,
                    p.model_year,
                    p.list_price
 from [production].[products] as p
 inner join [production].[brands]as b
 on p.brand_id = b.brand_id
 inner join [production].[categories] as c
 on p.category_id = c.category_id;

 select * from vw_bike_catalog
 where list_price >  2000
 order by list_price desc ;
```

Q16 BONUS: Create a view called vw_customer_orders showing: customer full name, order_id, order_date, store_name, and order_status. Then query it to show only orders where the customer city is "New York", sorted by order_date.

VIEWS

Your answer:

```
create view vw_customer_orders as
Select
                    c.first_name +' '+ c.last_name as Customer_Name,
                    o.order_id,
                    o.order_date,
                    s.store_name,
                    o.order_status
from [sales].[orders] as o
inner join [sales].[customers] as c
on o.customer_id = c.customer_id
inner join [sales].[stores] as s
on o.store_id = s.store_id;

select v.*  from vw_customer_orders v
inner join [sales].[customers] c
 on v.Customer_Name = c.first_name + ' ' + c.last_name
where city = 'New York'
order by order_date asc;
```